

Guía Avanzada y Reporte Técnico: Visualización de Curvas Implícitas en LaTeX

Autor: Departamento de Soporte Editorial Científico

Fecha: Julio de 2026

Tema: Computación Gráfica e Integración en TeX

Estado: Documento de Referencia Autónomo

1. Introducción y el Desafío Arquitectónico

El sistema de composición tipográfica **LaTeX** (basado en el motor original de Donald Knuth) fue diseñado específicamente para la maquetación de textos y fórmulas matemáticas complejas, no como un lenguaje de programación de propósito general enfocado en computación gráfica pesada. El trazado de una función explícita tradicional del tipo $y = f(x)$ requiere un muestreo lineal simple sobre el eje de las abscisas; un proceso trivial que librerías nativas como PGF/TikZ pueden resolver internamente mediante macros algebraicas básicas.

Sin embargo, la representación gráfica de una curva implícita definida por una restricción de la forma:

$$f(x, y) = 0$$

plantea un problema computacional radicalmente distinto. En la inmensa mayoría de los casos, es analíticamente imposible despejar una variable en función de la otra (por ejemplo, en curvas no algebraicas o polinomios de grado superior). El renderizado preciso exige la evaluación de un espacio bidimensional completo y la ejecución de algoritmos de búsqueda de raíces o extracción de isocontornos (tales como *Marching Squares* o subdivisiones adaptativas mediante *Quadtrees*).

Intentar implementar dichos algoritmos iterativos puramente mediante la sintaxis de macros de TeX penalizaría drásticamente los tiempos de compilación y provocaría desbordamientos de memoria de pila (*stack overflow*). Por esta razón, el ecosistema profesional de **LaTeX** delega el procesamiento matemático pesado a motores externos o subprocesos de lenguajes compilados, recuperando el resultado en forma de primitivas vectoriales nativas que garantizan una tipografía y resolución impecables.

2. Método 1: Pgfplots + Gnuplot (El Estándar Industrial)

La estrategia más extendida y robusta combina el paquete **pgfplots** con el motor de cálculo matemático externo **Gnuplot**. Gnuplot actúa como el backend de cálculo, procesando la ecuación y generando un archivo temporal de coordenadas que **pgfplots** lee de forma transparente para trazar la curva con las fuentes tipográficas del documento.

2.1 Mecanismo de Comunicación e Interfaz

Para que este método funcione, el compilador de LaTeX debe tener autorización para ejecutar programas externos. Esto se logra activando la bandera de escape del sistema (**shell escape**) al compilar:

```
pdflatex -shell-escape mi_documento.tex
```

Bajo el capó, *pgfplots* traduce la instrucción de LaTeX a una serie de comandos nativos de Gnuplot, invocando una instrucción de mapeo tridimensional de contornos planos (isolineas) donde la altura se fija exactamente en cero ($z = 0$).

2.2 Ejemplo de Código de Mapeo Completo (MWE)

El siguiente código muestra la implementación exacta para graficar el *Folium de Descartes*, definido por la ecuación implícita $x^3 + y^3 - 3xy = 0$:

```
\documentclass{article}
\usepackage{pgfplots}
\pgfplotsset{compat=1.18}

\begin{document}
\begin{figure}[ht]
\centering
\begin{tikzpicture}
\begin{axis}[
  xmin=-2.5, xmax=2.5,
  ymin=-2.5, ymax=2.5,
  axis lines=center,
  grid=both,
  xlabel={\mathit{x}},
  ylabel={\mathit{y}},
  title={Folium de Descartes:  $x^3 + y^3 - 3xy = 0$ },
  title style={font=\small\bfseries}
]
\addplot[
  thick,
  color=blue,
  raw gnuplot,
  id=folium,
  empty line=jump
] gnuplot {
  set contour base;
  set cntrparam levels discrete 0;
  unset surface;
  set table;
  splot x**3 + y**3 - 3*x*y;
};
\end{axis}
\end{tikzpicture}
\end{figure}
\end{document}
```

Nota Crítica sobre la Sintaxis de Gnuplot

Gnuplot procesa las potencias mediante el operador de doble asterisco (x^{**3}) en lugar del circunflejo estándar de LaTeX (x^3). Olvidar esta distinción sintáctica dentro del bloque `raw gnuplot` romperá silenciosamente el subproceso de cálculo.

3. Método 2: Asymptote (Alto Rendimiento e Infraestructura Vectorial)

Cuando se requiere graficar ecuaciones de alta complejidad topológica, superficies tridimensionales o ecuaciones implícitas simultáneas, **Asymptote** supera sustancialmente a TikZ. Asymptote es un lenguaje de gráficos vectoriales independiente con sintaxis basada en C++ que lee descripciones en formato de texto y genera salidas EPS, PDF o SVG de alta fidelidad.

3.1 Ventajas Algorítmicas

A diferencia de TikZ, Asymptote incorpora rutinas nativas avanzadas de álgebra lineal y análisis numérico. Su módulo dedicado `contour` implementa algoritmos adaptativos de subdivisión espacial extremadamente rápidos. No sufre de las limitaciones de memoria interna de TeX, permitiendo mallas de muestreo densas (p. ej., `500 \times 500` puntos) en milisegundos.

3.2 Ejemplo de Código Integrado

Para compilar este bloque, el paquete `asymptote` genera un archivo `.asy` intermedio que debe procesarse con el binario de Asymptote antes de realizar la pasada final de PDFLaTeX.

```

\documentclass{article}
\usepackage{asymptote}

\begin{document}
\begin{asy}
import contour;
import graph;

size(200);

// Definición de la función implícita  $f(x,y) = x^2 + y^2 - x^4 - y^4$ 
real f(real x, real y) {
    return x^2 + y^2 - x^4 - y^4;
}

real[] levels = {0}; // Buscamos  $f(x,y) = 0$ 
int grid_resolution = 150;

guide[][] c = contour(f, (-1.5,-1.5), (1.5,1.5), levels, grid_resolution);

// Trazado de los contornos extraídos
draw(c[0], darkblue + linewidth(1.2pt));

xaxis("$x$", BottomTop, LeftTicks);
yaxis("$y$", LeftRight, RightTicks);
\end{asy}
\end{document}

```

4. Método 3: LuaLaTeX + Lua (Solución Nativa sin Dependencias Externas)

Para entornos restringidos donde el uso de `-shell-escape` está prohibido por políticas de seguridad o donde no es posible instalar binarios adicionales como Gnuplot o Asymptote, la combinación de **LuaLaTeX** junto al paquete de gráficos TikZ representa la frontera moderna de la graficación matemática en TeX.

4.1 Arquitectura Lua-TeX

LuaLaTeX integra un intérprete completo del lenguaje de script Lua de forma nativa dentro del binario de compilación. Debido a que Lua es un lenguaje imperativo, ultraligero y de alta velocidad, es perfectamente viable programar un algoritmo de *Marching Squares* directo en el código fuente. Lua realiza el cómputo matricial puro en memoria flotante nativa de la CPU y devuelve una cadena de coordenadas estructurada directamente en comandos `\draw plot coordinates {...}`; comprensibles para TikZ.

4.2 Muestra de Implementación Algorítmica Autónoma

A continuación se expone la estructura de un script interno que evalúa una cuadrícula y genera el trazado de forma transparente al usuario:

```

\documentclass{article}
\usepackage{tikz}
\usepackage{luacode}

\begin{luacode*}
-- Función matemática a evaluar
function f(x, y)
    return x^2 + y^2 - 1.0 -- Ejemplo de circunferencia unitaria
end

function generar_curva_implicita(xmin, xmax, ymin, ymax, steps)
    local hx = (xmax - xmin) / steps
    local hy = (ymax - ymin) / steps

    -- Algoritmo simplificado de escaneo de inversión de signo
    for i = 0, steps - 1 do
        local x = xmin + i * hx
        for j = 0, steps - 1 do
            local y = ymin + j * hy

            local v0 = f(x, y)
            local v1 = f(x + hx, y)
            local v2 = f(x, y + hy)

            -- Detectar cruce de contorno elemental
            if (v0 * v1 < 0) or (v0 * v2 < 0) then
                tex.sprint(string.format("\draw[red, fill] (%.4f,%.4f) circle (0.02);",
x, y))
            end
        end
    end
end
\end{luacode*}

\begin{document}
\begin{tikzpicture}[scale=2]
    \draw[-latex] (-1.5,0) -- (1.5,0) node[right] {$x$};
    \draw[-latex] (0,-1.5) -- (0,1.5) node[above] {$y$};
    % Invocación directa del motor numérico de Lua
    \directlua{generar_curva_implicita(-1.2, 1.2, -1.2, 1.2, 80)}
\end{tikzpicture}
\end{document}

```

5. Comparativa Técnica y Matriz de Decisión

Cada alternativa metodológica introduce un conjunto específico de compromisos o *trade-offs* de ingeniería de software científico. La siguiente matriz resume los criterios fundamentales de evaluación:

Criterio Metodológico	Pgfplots + Gnuplot	Asymptote	LuaLaTeX Integrado
Rendimiento de Cómputo	Alto. Delegado a binario compilado en C.	Excelente. Diseñado para optimización de mallas.	Medio-Alto. Ejecución interpretada JIT/Lua.
Portabilidad del Código	Baja. Requiere Gnuplot instalado en el PATH.	Baja. Requiere cadena de herramientas de compilación extra.	Absoluta. Funciona en cualquier distribución LaTeX moderna.
Seguridad de Compilación	Riesgosa. Requiere apertura de shell-escape.	Segura (Procesado de archivos estáticos independientes).	Máxima. No requiere permisos de ejecución externos.
Fidelidad Tipográfica	Perfecta. Conserva los estilos de fuente globales nativos.	Excelente. Soporta renderizado de etiquetas vía LaTeX.	Perfecta. Genera nodos y primitivas TikZ puras.
Curva de Aprendizaje	Media. Doble sintaxis (LaTeX / Gnuplot).	Alta. Requiere aprender un lenguaje cuasi C++.	Alta. Requiere conocimientos de programación algorítmica.

6. Conclusiones y Recomendaciones Profesionales

La selección del pipeline técnico óptimo depende estrictamente del entorno editorial del proyecto:

- Proyectos Académicos Estándar (Tesis, Artículos de Revistas):** Se recomienda enfáticamente la opción **Pgfplots + Gnuplot**. Es el estándar de la industria, las plantillas de las editoriales científicas suelen permitirlo y asegura que los textos del gráfico cambien de peso o fuente al unísono con el cuerpo del documento.
- Entornos de Servidores Autónomos o Compilación en la Nube (Overleaf sin permisos, APIs web):** La aproximación vía **LuaLaTeX** representa la única vía robusta para ofrecer generación dinámica en tiempo real garantizando el aislamiento del sistema de archivos.
- Monografías de Alta Densidad Geométrica o Libros de Cálculo Vectorial Avanzado:** La potencia tipográfica y el manejo de matrices de **Asymptote** justifican la complejidad adicional en la cadena de compilación de desarrollo.